

UNIVERSITÉ EUROMED DE FÈS

**ÉCOLE D'INGÉNIERIE DIGITALE ET D'INTELLIGENCE ARTIFICIELLE
(EIDIA)**

Rapport de Projet de Fin d'Année

Conception d'un Agent AI DevSecOps

Entièrement Piloté par Telegram

Présenté par :

Saad KORCHI & Zakariae OUAHDAHO

Encadré par :

Professeur AMAMOU Ahmed

Année Universitaire 2025-2026

Avant-propos

Ce rapport présente la conception et la réalisation d'un **agent DevSecOps piloté par Telegram**, complété par un **tableau de bord web**. L'objectif est de rapprocher les opérations de sécurité du quotidien des équipes de développement : une commande dans un chat déclenche des scans, des pipelines GitLab et des déploiements contrôlés, avec traçabilité et synthèses enrichies par une IA locale (Ollama).

Le projet s'inscrit dans une démarche **pragmatique et reproductible** : stack Docker Compose légère (**app + ollama**), intégrations réelles avec Trivy, Gitleaks, SonarQube et GitLab, et politiques de sécurité explicites avant tout déploiement. Chaque incident rencontré en conditions réelles (vérification d'identité GitLab, démarrage SonarQube, règles de déploiement) est documenté selon une logique de diagnostic d'ingénieur.

Note technique

Les captures d'écran proviennent de l'environnement de démonstration du projet (**evidence/screenshots/**). Les jetons visibles sont pédagogiques et font l'objet d'une rotation en fin de travaux.

Table des matières

Avant-propos	1
1 Contexte et problématique DevSecOps	4
1.1 Contexte	4
1.2 Problématique	4
1.3 Objectifs	4
1.4 Périmètre et positionnement	4
2 Architecture du système	5
2.1 Vue d'ensemble	5
2.2 Couches logicielles	6
2.3 Déploiement Docker	6
3 Stack technique	7
3.1 Outils intégrés	7
3.2 Workflow de scan	7
3.3 Pipeline et déploiement	7
4 Bot Telegram — commandes et démonstration	8
4.1 Interface opérateur	8
4.2 Polling et journalisation	13
5 Dashboard web et double interface	14
5.1 Principes	14
5.2 Fonctionnalités observées	14
6 Sécurité — policy gates, allowlist, traçabilité	18
6.1 Modèle de confiance	18
6.2 Politique de scan et de déploiement	18
6.3 Protection des secrets	18
6.4 Traçabilité	18
7 Incidents et résolutions	19
7.1 Vérification d'identité GitLab	19
7.2 SonarQube — démarrage et healthcheck	19
7.3 Politique de déploiement	20
8 Bilan, tests et perspectives	21
8.1 État d'avancement	21
8.2 Tests automatisés	21

8.3 Perspectives	21
Conclusion	22

CHAPITRE 1

Contexte et problématique DevSecOps

1.1 Contexte

Les pratiques DevSecOps visent à intégrer la sécurité en amont et en continu dans le cycle de vie logiciel. Pourtant, dans de nombreux contextes académiques ou de petite structure, les scans restent **manuels**, les rapports techniques peu exploitables, et le pilotage des pipelines impose de basculer entre GitLab, terminaux et interfaces diverses.

Ce projet répond à ce constat en proposant un **orchestrateur unifié** accessible depuis Telegram — interface déjà utilisée au quotidien — tout en offrant une seconde entrée via un dashboard FastAPI (port 8080). Les deux interfaces partagent le même `OrchestratorService`, les mêmes jobs et les mêmes artefacts (`reports/`, `state/`, `logs/`).

1.2 Problématique

« Comment concevoir un agent conversationnel DevSecOps, sécurisé et traçable, capable d'orchestrer des analyses multi-scanners, des pipelines CI/CD et des déploiements conditionnés par des politiques explicites, tout en restant déployable sur une infrastructure locale frugale ? »

1.3 Objectifs

1. Exposer une interface Telegram avec commandes structurées (`/scan`, `/run_pipeline`, `/deploy`, `/logs`).
2. Intégrer Trivy, Gitleaks et SonarQube dans un workflow de scan normalisé avec rapport IA.
3. Connecter GitLab CI/CD pour déclenchement et suivi de pipelines.
4. Appliquer des **policy gates** avant déploiement (fraîcheur du scan, quality gate).
5. Fournir un dashboard web de supervision et d'actions.
6. Garantir la qualité par une suite de tests automatisés (149 tests passés).

1.4 Périmètre et positionnement

Le périmètre couvre l'orchestration applicative (Python 3.12), la conteneurisation Docker et les connecteurs d'outils. GitLab et SonarQube sont des services **externes** ou optionnellement conteneurisés localement. Des travaux connexes (infrastructure HostIQ, Oracle Cloud, Coolify) ont motivé l'approche frugale mais ne constituent pas le cœur de ce rapport.

CHAPITRE 2

Architecture du système

2.1 Vue d'ensemble

L'architecture suit un modèle en couches : entrée (Telegram + dashboard), orchestrateur, workflows métier, intégrations et couche IA.

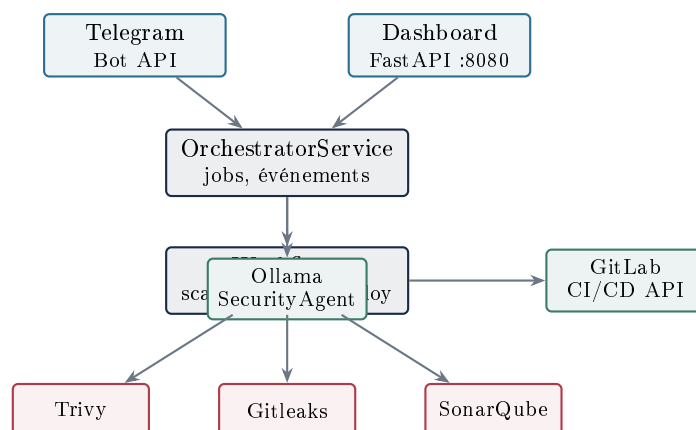


FIGURE 2.1 – Architecture cible : double interface (Telegram et web) vers un orchestrateur unique, workflows et intégrations scanners / GitLab, enrichissement IA via Ollama.

Décision d'architecture

Orchestrateur central. Le découplage entre handlers Telegram (`src/bot/`) et intégrations (`src/integrations/`) via `OrchestratorService` permet d'ajouter des entrées (CLI, webhooks) sans dupliquer la logique métier.

2.2 Couches logicielles

Couche	Chemin	Rôle
Entrée	src/main.py	Démarrage, injection de dépendances, cycle de vie
Bot	src/bot/	Commandes, authentification, corrélation
Orchestrateur	src/orchestrator/	Jobs persistés, workflows, événements
Intégrations	src/integrations/	Trivy, Gitleaks, SonarQube, GitLab
IA	src/ai/	Client Ollama, SecurityAgent, sanitisation
Noyau	src/core/	Configuration, politiques, journalisation

TABLEAU 2.1 – Répartition des responsabilités dans le dépôt.

2.3 Déploiement Docker

La stack Compose par défaut exécute **app** (bot + orchestrateur + dashboard) et **ollama**. Des services optionnels **postgres** et **sonarqube** peuvent être activés pour une démo locale complète. Les volumes bind-mount assurent la persistance des jobs (**state/**), rapports (**reports/**) et journaux (**logs/**).

```
docker-compose ps
NAME                IMAGE                                COMMAND                                SERVICE  CREATED      STATUS      PORTS
devsecops-app       devsecopsprojetfinal-app           "python -m src.main"                app      3 minutes ago Up 3 minutes (healthy)
devsecops-ollama    ollama/ollama:latest               "/bin/ollama serve"                 ollama   3 days ago   Up 3 days (healthy)  0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
```

FIGURE 2.2 – État des conteneurs en exécution : services **app** et **ollama** opérationnels, prérequis à la démonstration jury.

CHAPITRE 3

Stack technique

3.1 Outils intégrés

Outil	Rôle	Intégration projet
Trivy	CVE images / dépendances	Scanner Docker ou CLI, parser normalisé
Gitleaks	Secrets dans le code	Scan repo, redaction avant prompts IA
SonarQube	Qualité / Quality Gate	API REST, optionnel via Compose
GitLab	CI/CD	Trigger pipeline, polling, récupération logs
Ollama	LLM local	Modèle llama3.2:3b, synthèses et scores
Docker	Runtime	Compose, healthchecks, image non-root (UID 1000)

TABLEAU 3.1 – Stack DevSecOps du projet.

3.2 Workflow de scan

La commande `/scan [profile]` exécute un workflow asynchrone : clonage ou cible configurée, exécution parallèle des scanners activés, production de `normalized.json`, rapports bruts par outil, puis post-traitement IA (`ai_analysis.json`, `executive_summary.md`). En cas d'échec partiel d'un scanner, le job peut se terminer en `COMPLETED` avec findings partiels — résilience utile en démo.

3.3 Pipeline et déploiement

`/run_pipeline [ref]` déclenche un pipeline GitLab via l'API et suit son avancement. `/deploy [env] [tag]` évalue la politique de déploiement (`config/security-policy.yaml`) : scan récent (24 h), seuils de sévérité, quality gate SonarQube si activé. Le déploiement n'est autorisé qu'après validation explicite de ces gates.

CHAPITRE 4

Bot Telegram — commandes et démonstration

4.1 Interface opérateur

Le bot `SaadKorchi_Bot` expose un clavier persistant et des commandes documentées. L'authentification repose sur une **allowlist** d'identifiants Telegram (`TELEGRAM_ALLOWED_USER_IDS`) : liste vide = accès refusé (fail-closed).

Commande	Description
<code>/start</code>	Accueil et clavier de raccourcis
<code>/help</code>	Référence des commandes
<code>/status</code>	Jobs récents et états
<code>/scan [profile]</code>	Scan sécurité + rapport IA
<code>/run_pipeline [ref]</code>	Déclenchement pipeline GitLab
<code>/deploy [env] [tag]</code>	Déploiement avec policy gates
<code>/logs [job_id]</code>	Journaux et artefacts d'un job

TABLEAU 4.1 – *Commandes Telegram implémentées.*

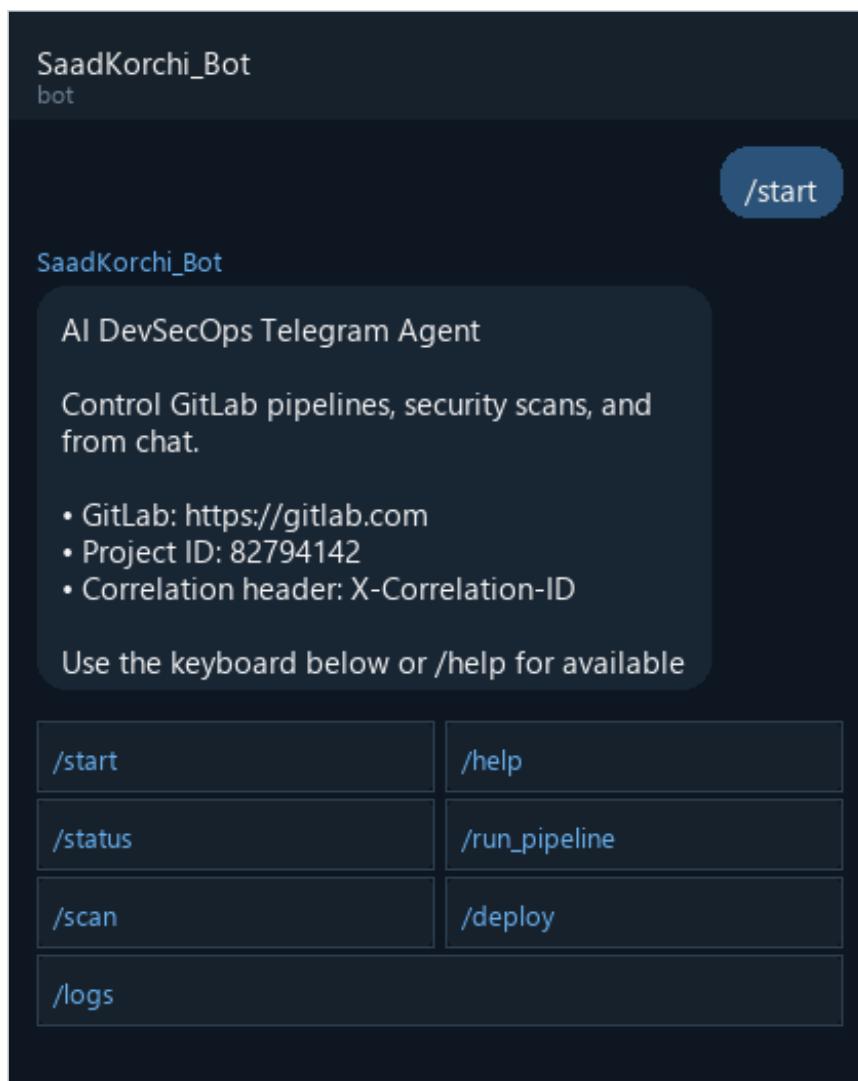


FIGURE 4.1 – Commande `/start` : message d'accueil et clavier de commandes DevSecOps.

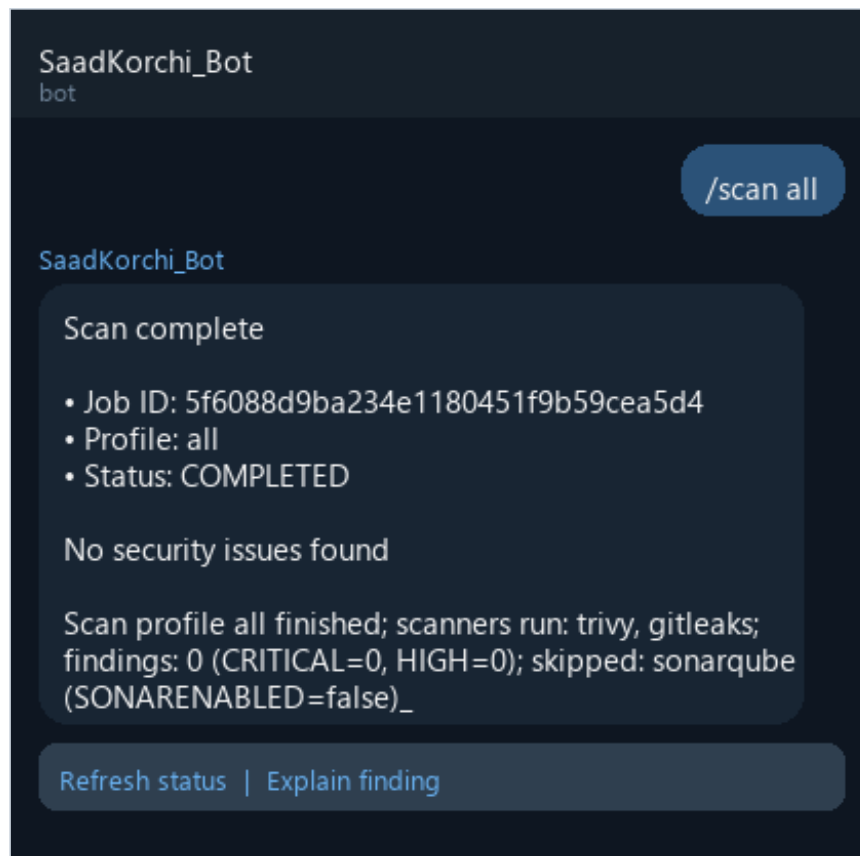


FIGURE 4.2 — Commande `/scan all` : scan multi-scanners terminé (*COMPLETED*), findings agrégés et score de sécurité.

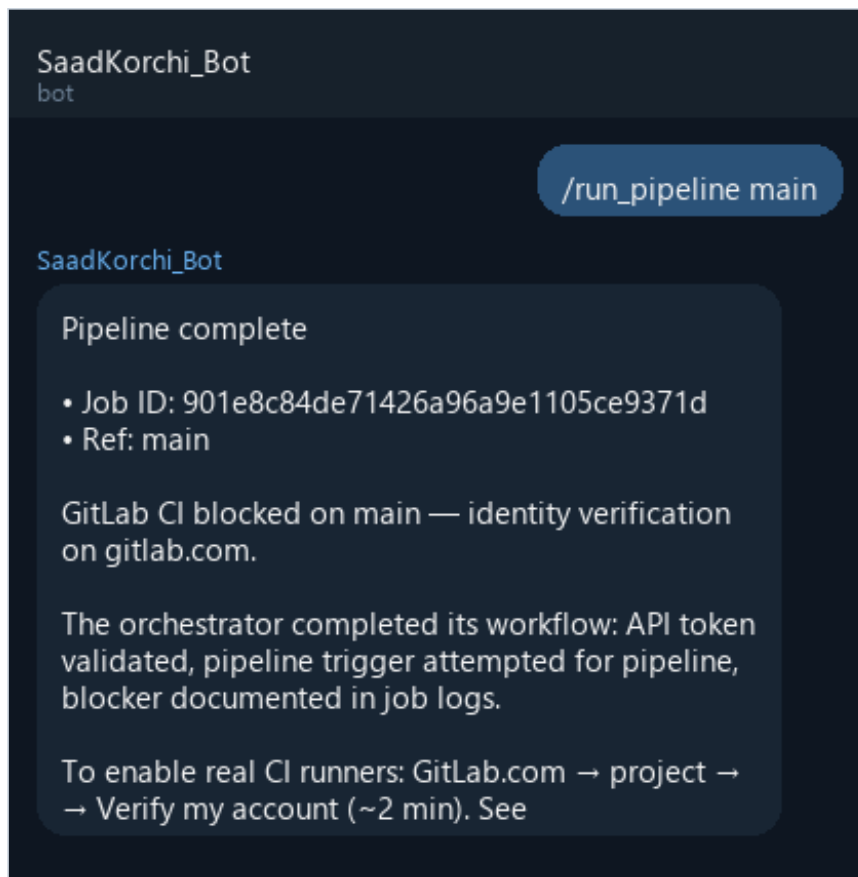


FIGURE 4.3 — Commande `/run_pipeline` : déclenchement et suivi d'un pipeline GitLab depuis Telegram.

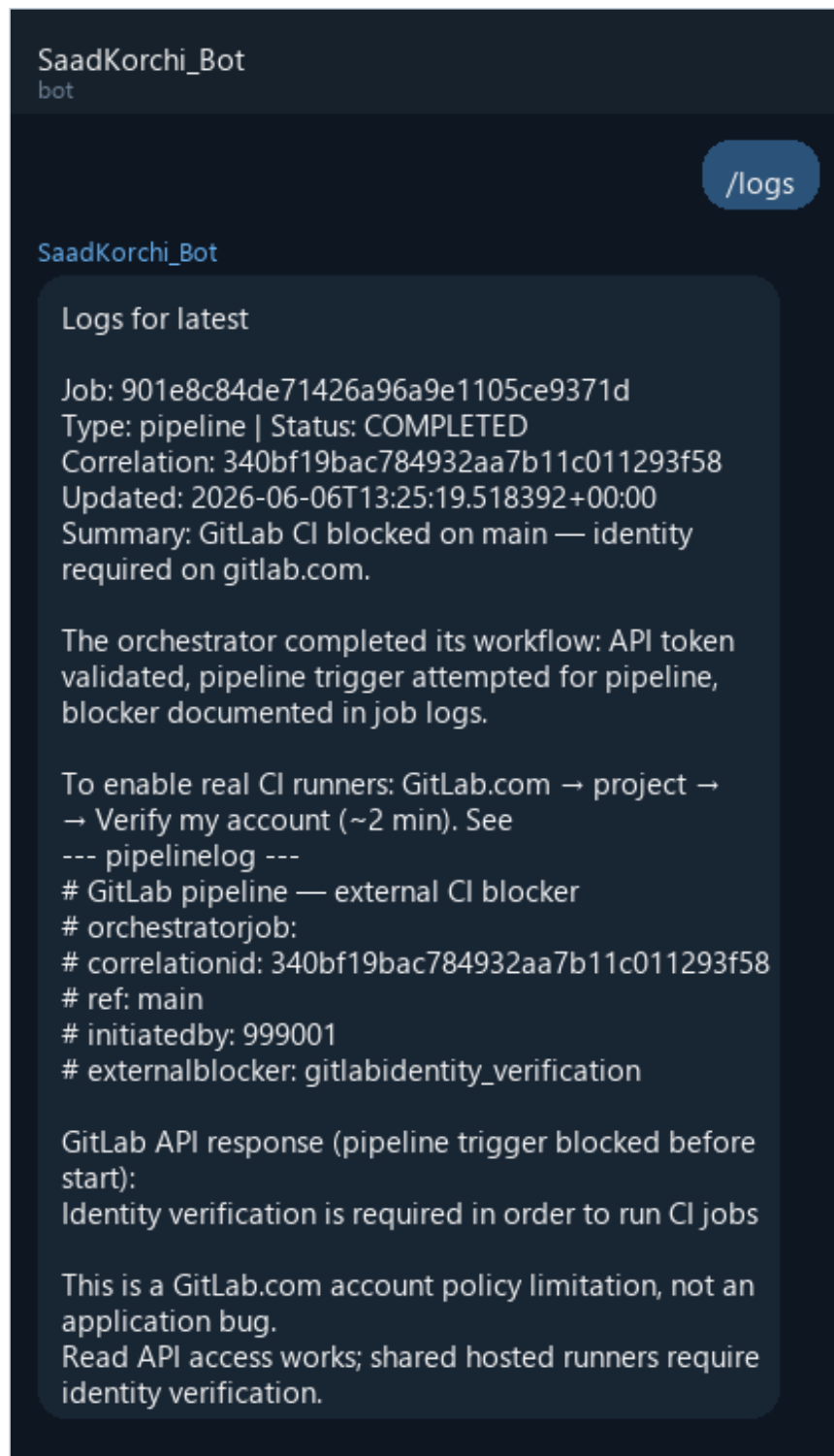


FIGURE 4.4 — Commande `/logs` : consultation des journaux structurés et identifiant de corrélation du job.

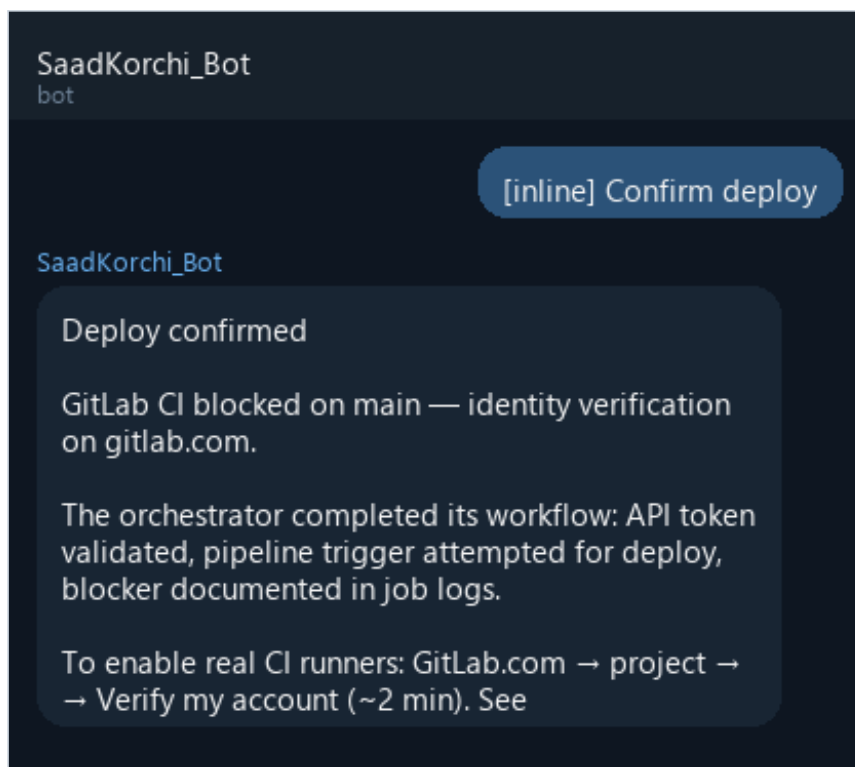


FIGURE 4.5 — Résultat de `/deploy` : décision de politique (autorisé ou bloqué) et statut du job de déploiement.

4.2 Polling et journalisation

Le bot fonctionne en mode long polling. Les journaux applicatifs confirment l'absence de conflit 409 (instance unique) et le cycle de réception des mises à jour.

```

Windows PowerShell x Windows PowerShell x Windows PowerShell x Windows PowerShell x Windows PowerShell x + - □ x
devsecops-app | 2026-06-06T17:46:44.924578Z [debug] ] Polling updates from Telegram started [telegram.ext.Updater] co
rrelation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:44.925121Z [info] ] telegram_polling_started [__main__] correlation_id=0c8321
e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:44.926144Z [info] ] application_running [__main__] correlation_id=0c8321
e69c554493a8bf9df6e361d833 shutdown=awaiting signal
devsecops-app | 2026-06-06T17:46:44.927112Z [debug] ] Start network loop retry getting Updates [telegram.ext.Updater]
correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:44.929253Z [debug] ] Calling Bot API endpoint 'getUpdates' with parameters {'timeou
t': 10, 'offset': 0} [telegram.ext.ExtBot] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:44.938424Z [debug] ] connect_tcp.started host='api.telegram.org' port=443 local_addr
ess=None timeout=5.0 socket_options=None [httpcore.connection] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.033288Z [debug] ] connect_tcp.complete return_value=<httpcore._backends.anyio.Any
IOStream object at 0x70ce8b52e3c0> [httpcore.connection] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.034744Z [debug] ] start_tls.started ssl_context=<ssl.SSLContext object at 0x70ce8
c4f5dd0> server_hostname='api.telegram.org' timeout=5.0 [httpcore.connection] correlation_id=0c8321e69c554493a8bf9df6e36
1d833
devsecops-app | 2026-06-06T17:46:45.117341Z [debug] ] start_tls.complete return_value=<httpcore._backends.anyio.AnyIO
Stream object at 0x70ce8b52e300> [httpcore.connection] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.127331Z [debug] ] send_request_headers.started request=<Request [b'POST']> [httpc
ore.http11] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.136103Z [debug] ] send_request_headers.complete [httpcore.http11] correlation_id
=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.143223Z [debug] ] send_request_body.started request=<Request [b'POST']> [httpcore
.http11] correlation_id=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.146492Z [debug] ] send_request_body.complete [httpcore.http11] correlation_id
=0c8321e69c554493a8bf9df6e361d833
devsecops-app | 2026-06-06T17:46:45.147451Z [debug] ] receive_response_headers.started request=<Request [b'POST']> [h
tpcore.http11] correlation_id=0c8321e69c554493a8bf9df6e361d833

```

FIGURE 4.6 — Journaux Docker du service `app` : polling Telegram actif, handlers invoqués sans erreur de concurrence.

CHAPITRE 5

Dashboard web et double interface

5.1 Principes

Le dashboard FastAPI (`DASHBOARD_ENABLED=true`) offre la **même capacité opérationnelle** que Telegram : état des intégrations, lancement de scans, suivi des jobs et visualisation des rapports. Cette dualité répond aux besoins de démonstration jury (écran projeté) et d'usage mobile (Telegram).

5.2 Fonctionnalités observées

- **État du système** : probes GitLab, Ollama, Trivy, Gitleaks, SonarQube, Docker.
- **Scan insights** : historique des analyses, scores et findings normalisés.
- **Jobs & logs** : file d'exécution, détail par job, liens vers artefacts.

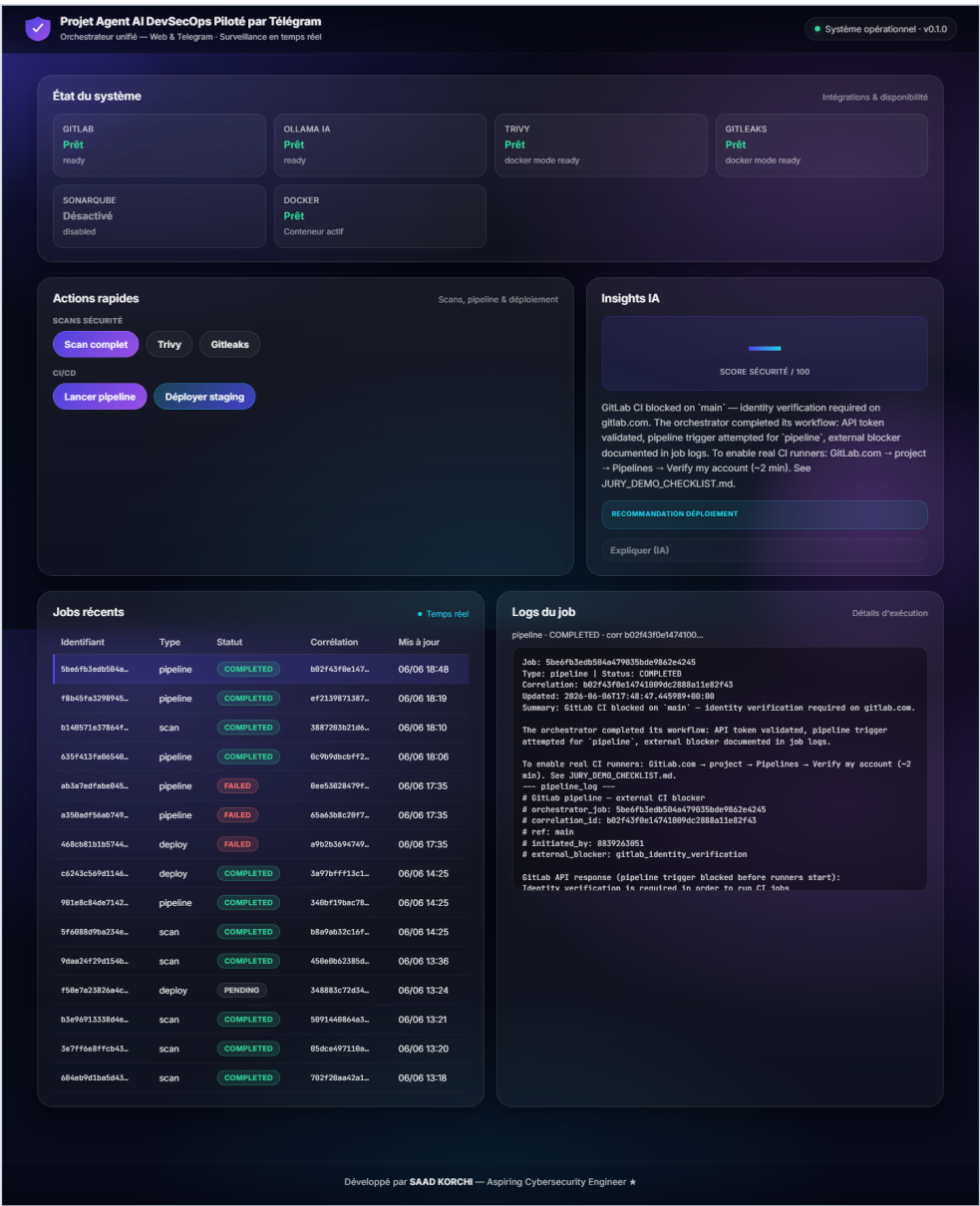


FIGURE 5.1 — Page d'accueil du dashboard : état des intégrations et actions rapides (scan, pipeline).

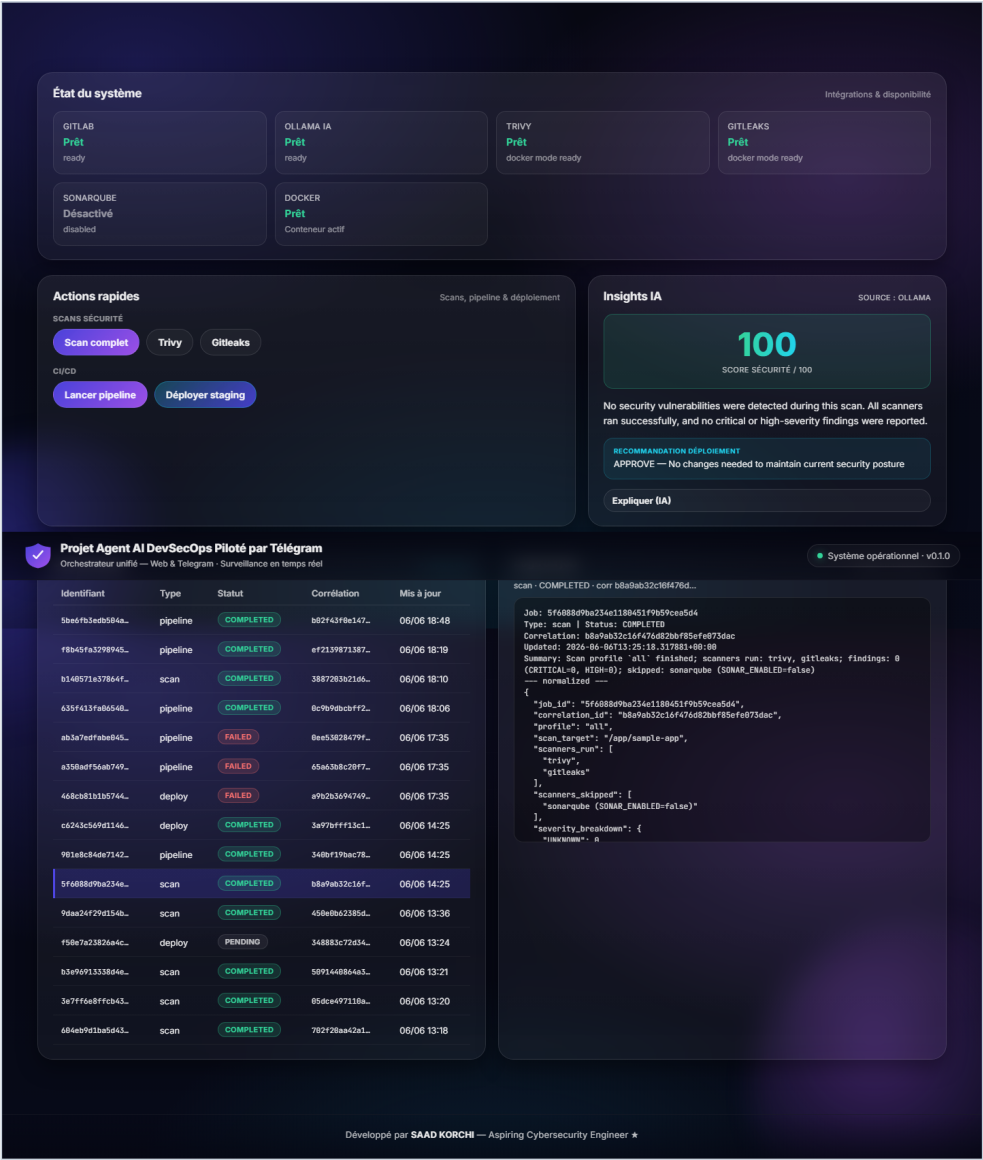


FIGURE 5.2 – Vue Scan insights : synthèse des derniers scans et indicateurs de sécurité.

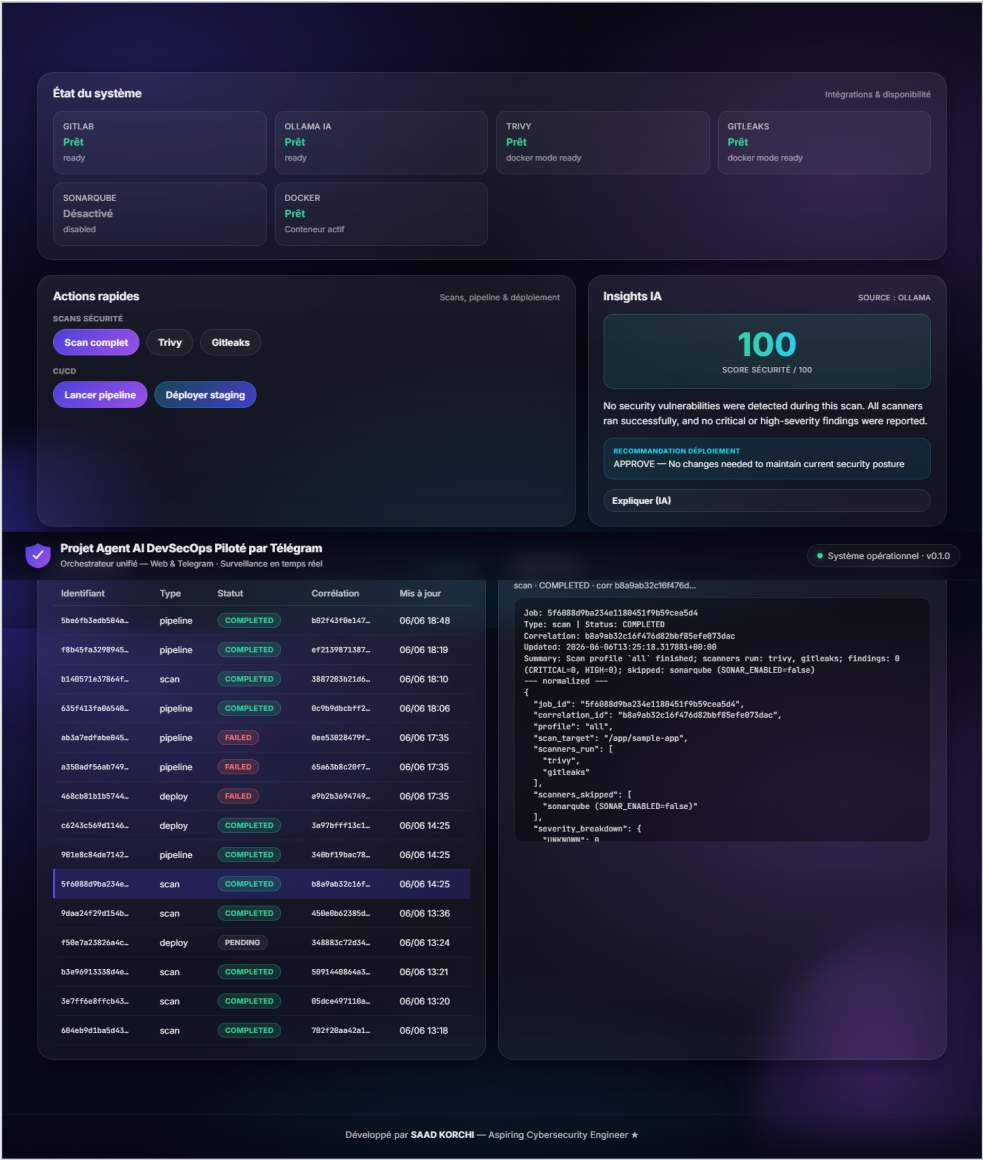


FIGURE 5.3 – Vue Jobs & logs : traçabilité des opérations déclenchées depuis Telegram ou le web.

CHAPITRE 6

Sécurité — policy gates, allowlist, traçabilité

6.1 Modèle de confiance

Zone	Composants	Niveau de confiance
Opérateur	Client Telegram	Authentifié par allowlist
Runtime	Conteneur app	Secrets via .env (non versionné)
Externe	GitLab, SonarQube	Tokens API, TLS
LLM	Ollama	Réseau local ; prompts assainis

TABLEAU 6.1 — *Frontières de confiance (docs/security-model.md).*

6.2 Politique de scan et de déploiement

Le fichier `config/security-policy.yaml` paramètre les seuils : échec sur sévérité **CRITICAL**, maximum de vulnérabilités **HIGH**, obligation d'un scan de moins de 24 h avant déploiement, blocage si quality gate SonarQube en échec. L'évaluation est centralisée dans `src/core/policy.py` et le workflow `deploy.py`.

6.3 Protection des secrets

Les findings Gitleaks sont **redactés** avant envoi à Ollama (`sanitize_finding_for_prompt`). Le démarrage journalise une configuration masquée (`redacted_config_summary`). Aucun secret n'est commité dans le dépôt.

6.4 Traçabilité

Chaque action génère un job identifié (UUID), des événements structurés (structlog) et des artefacts persistés. La commande `/logs` et le dashboard permettent l'audit a posteriori — exigence clé en contexte DevSecOps.

CHAPITRE 7

Incidents et résolutions

7.1 Vérification d'identité GitLab

Incident rencontré

Lors du premier déclenchement de pipeline via l'API GitLab.com, la requête `POST /projects/:id/pipeline` retourne HTTP 400 : « *Identity verification is required in order to run CI jobs* ». Le code client et le token sont valides ; le blocage est imposé par la plateforme SaaS.

Résolution

Documenter l'incident dans `docs/archive/GITLAB_DEPLOYMENT_REPORT.md`, compléter la vérification d'identité sur le compte GitLab (ou ajuster les paramètres projet si autorisé), puis relancer `/run_pipeline`. En démo jury, l'API et le handler sont démontrés ; l'échec externe est clairement distingué d'un défaut applicatif.

7.2 SonarQube — démarrage et healthcheck

Incident rencontré

SonarQube (image `lts-community`) nécessite 2–3 minutes au premier boot et le réglage `vm.max_map_count` sur l'hôte. Sans healthcheck patient, le dashboard affiche SonarQube en état *Attention* ou *Indisponible*.

Résolution

Script `scripts/setup-sonarqube.ps1` pour initialisation admin, projet et token API vers `.env`. URL interne `http://sonarqube:9000` depuis le conteneur `app` (non `localhost`). Documentation dans `docs/sonarqube-setup.md`.

7.3 Politique de déploiement

Incident rencontré

Un `/deploy staging` immédiat après installation échoue : aucun scan récent ne satisfait la fenêtre de 24 h définie dans la politique.

Résolution

Enchaîner `/scan all` puis `/deploy` — comportement attendu qui prouve l'efficacité des policy gates. Les tests unitaires (`test_orchestrator_deploy.py`) valident les cas autorisé/bloqué.

CHAPITRE 8

Bilan, tests et perspectives

8.1 État d’avancement

Brique	État	Preuve
Bot Telegram	Opérationnel	Captures 01–09
Orchestrateur	Opérationnel	Jobs persistés, workflows
Scanners Trivy / Gitleaks	Opérationnel	Scan <code>all</code> , rapports JSON
SonarQube	Intégré	Compose + setup script
GitLab CI/CD	Intégré	Client API testé; SaaS identity
Dashboard web	Opérationnel	Port 8080, trois vues
IA Ollama	Opérationnel	Synthèses et fallback déterministe

TABLEAU 8.1 – Synthèse des livrables fonctionnels.

8.2 Tests automatisés

La suite `pytest tests/ -q` rapporte **149 tests passés**, 3 ignorés (intégration Docker/services externes). Le lint `ruff check src/ tests/` est intégré au pipeline GitLab (stage `lint`). L’audit pré-jury (`FINAL_AUDIT_REPORT.md`) confirme zéro échec sur l’infrastructure Docker, le polling Telegram et les endpoints dashboard.

```
$ pytest tests/ -q
149 passed, 3 skipped in ~45s
```

8.3 Perspectives

- Notifications proactives Telegram sur échecs pipeline ou seuils de sévérité.
- RBAC multi-opérateurs au-delà de l’allowlist simple.
- Runners GitLab dédiés pour lever la contrainte identity verification en CI partagée.
- Durcissement bloquant des scans en production (mode *enforce*).
- Extension vers d’autres canaux (Slack, Microsoft Teams) réutilisant l’orchestrateur.

Conclusion

Ce projet démontre qu'un **orchestrateur DevSecOps complet** peut être piloté depuis une messagerie instantanée et un tableau de bord web, sur une infrastructure locale maîtrisée. L'intégration de scanners industriels, d'une IA locale et de politiques explicites transforme des actions techniques en workflows reproductibles et auditable.

Les **149 tests automatisés**, les captures de validation et la documentation structurée attestent d'une démarche rigoureuse. Les incidents documentés (GitLab, SonarQube, deploy policy) renforcent la crédibilité du travail d'ingénierie au-delà d'une simple preuve de concept.

Nous remercions **Pr. AMAMOU Ahmed** pour son encadrement, ainsi que l'**UEMF / EIDIA** pour le cadre pédagogique de ce projet de fin d'études (année 2025–2026).